

OKAR Architecture Cheat Sheet

One sentence

****OKAR is a node-local runtime that turns package claims into inspectable protocol routes, preserves durable evidence, and exchanges exact bytes under local trust policy.****

Layers

Layer	Responsibility
Operator / CLI	Installs packages; inspects routes; sets local trust and route policy.
Runtime / kernel	Activates packages, derives routes, validates known records, manages relay and workflow lifecycle.
Packages	Own domain commands, protocol families, validators, and implementation claims.
Durable stores	Append-only history plus CAS for immutable payloads and lifecycle events.
Grid / relay	Moves signed records and metadata; discovery does not create trust.

Key flow

```
package manifest + self-check
    ↓
claims → runtime route table → inspect / plan / policy
    ↓
known record → package validation → append-only history
unknown record → exact-byte relay retention
    ↓
workflow artifact → CAS lifecycle event → disposable local projection
```

Important boundaries

- ****Protocol identity:**** pCID identifies a protocol; it is not a message hash.
- ****Trust:**** peer discovery is not permission to pull or push.
- ****Authority:**** importing a workflow artifact does not by itself grant worker or route-execution authority.
- ****Durability:**** CAS events and artifacts are authoritative; caches are not.
- ****Current limit:**** routing is planned and explained, but not yet a complete promise-based multi-agent dispatch system.

Source: DI-moksu, DI-puvok, DI-bavuk.